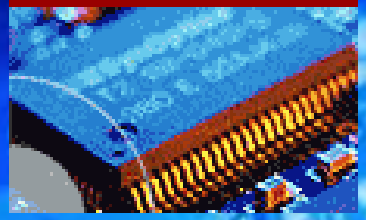
EEE3096S



Embedded Systems II

ARM
Advanced RISC Machines

Part2/2

ARM AArch32 Architecture Fundamentals

Embedded Systems II

L07

Dr Simon Winberg

Electrical Engineering
University of Cape Town

Outline of Lecture

- ARM Naming Convention, ARM Cortex
- ARM AArch32 Architecture Fundamentals
  - Data & Instructions
  - 7 Modes of operation
  - States of operation
  - ARM Registers (PSR, PC)
  - Exception handling
- ARM's endianness switchianess

ARM

Advanced RISC Machines

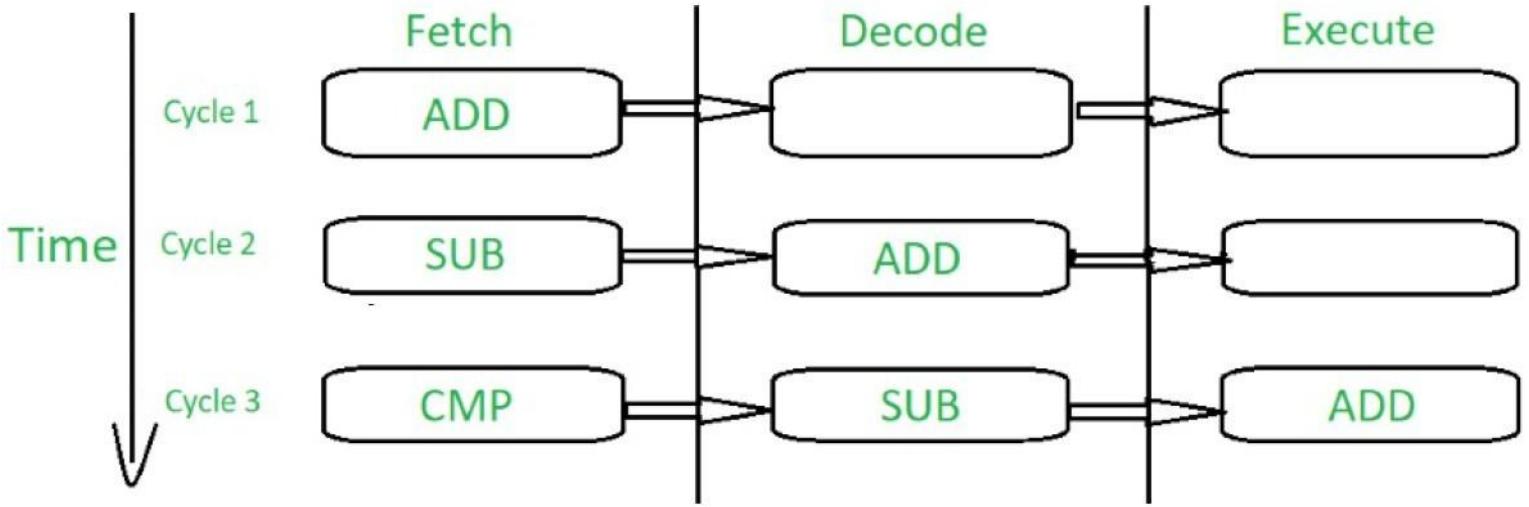
Our focus: ARM Cortex

- ARM Cortex range introduced in 2005
- Key features of the Cortex-M0:
  - ARMv6-M architecture
  - High performance relative to power consumption
  - Some Thumb* instruction set support
  - 3-stage pipeline

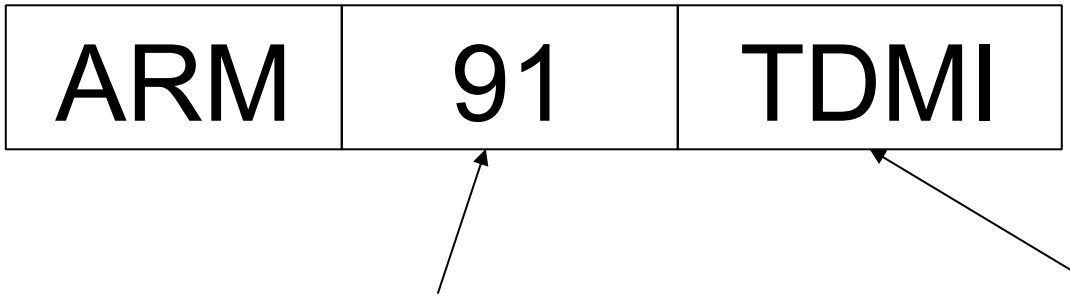
* Covered soon …

ARM Cortex 3-Stage Pipeline

- Stages: Fetch, Decode, Execute



ARM Core Naming Convention



Version of the Core. (91 is family 9, version 1)

Extensions

Note that ARM10 and ARM11 are separate families from ARM1

ARM Extensions

ARM	91	TDMI

Some of the extensions:

E DSP instruction extensions

J Jazelle, Java bytecode extensions

S Softcore – provided as synthesizable VDHL or Verilog code

T Thumb – 16 bit operations supported

Why Thumb Instructions?

- Thumb halves size of the instructions; uses 16-bit instructions on 32-bit system
- System can load two Thumb instructions in one data word and process them in correct order
- **But** this at expense of complexity/functionality of the instruction set (i.e. less instructions to choose from)
- For many applications, code density can be nearly doubled using Thumb instructions, which can significantly increasing performance

ARM32 vs ARM64

- You might assume ARM64 uses double the address line and can still run Thumb code …
- BUT you'd be mistaken about that. The designers for ARM64 were being realistic and sensible.
- Generally ARM64 *can* swap to ARM32 but can't swap mode in one execution block (should be separate 'basic blocks' at least)
- Cannot mix 64-bit and 32-bit instructions within the same exact execution thread or mode.
- To swap, need to execute an exception and change of mode like an SVC or interrupt

SVC = 'Supervisor Call' (not as simple as SWI)

SWI = Software Interrupt

ARM64 vs ARM32

- ARM64 vs ARM32Instruction Width:
  - ARM32 mixes 32-bit (ARM) and variable 16/32-bit (Thumb/Thumb-2) instructions.
  - ARM64 uses a clean, fixed 32-bit (4-byte) instruction width everywhere.
- No Thumb Mode: ARM64 has no concept of “Thumb state” or switching execution modes to Thumb (but it can switch to ARM32 and then to using Thumb).
- Registers:
  - ARM32 has 16 general-purpose registers.
  - ARM64 increases this to 31 general-purpose 64-bit registers (X0–X30), which can also be accessed as 32-bit registers (W0–W30).

Reassuring Point!

In this course you are not expected to know ARM64, no need to know how to switch to that state and characteristics of ARM64.

But know some of the basics about ARM64, e.g. main points mentioned in previous two slides.

AArch32
&
Programming the ARM 32-bit Architecture

Data and Instructions

- ARM 32-bit architecture (termed ‘AArch32’)
- Support operations on bytes (8-bit), half-words (16-bit) and words (32-bit).
- All cores support 32-bit ARM instructions
- Most cores (ARM7 and later) support 16-bit Thumb instructions
- Some (Jazelle) cores support 8-bit Java bytecode

ARM Mode of Operation

- A processor (e.g. ARM Cortex) can have multiple ‘modes of operation’.
- A mode of operation is a
  - distinct operating state or configuration that specifies the processor’s privileges, access to system resources, and the types of instructions it can execute.
- Modes of operation are crucial for maintaining system security, stability, and managing levels of software execution

ARM processors generally have 7 modes of operation …

The 7 modes of operation

1. User Mode
   - Restricted mode – limitations on register access and memory addressing.
   - Applications generally run in this mode.
2. Interrupt ReQuest (IRQ)
   - This mode is used when a low priority interrupt is signalled.

The 7 modes of operation

3. Fast Interrupt reQuest (FIQ)
   - Used when a high priority interrupt is signalled
   - Handled differently to a regular IRQ
4. Supervisor
   - Entered on system reset and on software interrupt instruction
5. Abort
   - Memory exceptions handled in this mode

The 7 modes of operation

6. Undef
   - Handles undefined instructions
7. System Mode
   - Privileged mode. Similar to user mode but with fewer restrictions.
   - Used primarily by OS kernel code.

The 4 states of operation

- ARM cores offer up to 4 states of operation:
  - ARM64 state (64 bit addressing*, registers)
  - ARM32 state (32 bit addressing, registers)
  - Thumb state (16 bit instructions)
  - Jazelle state (8 bit Java bytecode)
- Trade-offs between speed and memory between the states

* The embedded platform is probably not going to have 2^64 addresses, i.e. a few million terrabypes of memory… not all the lines are likely to be hooked to address lines, but it could be useful in e.g. virtual addressing, disk access, etc etc.

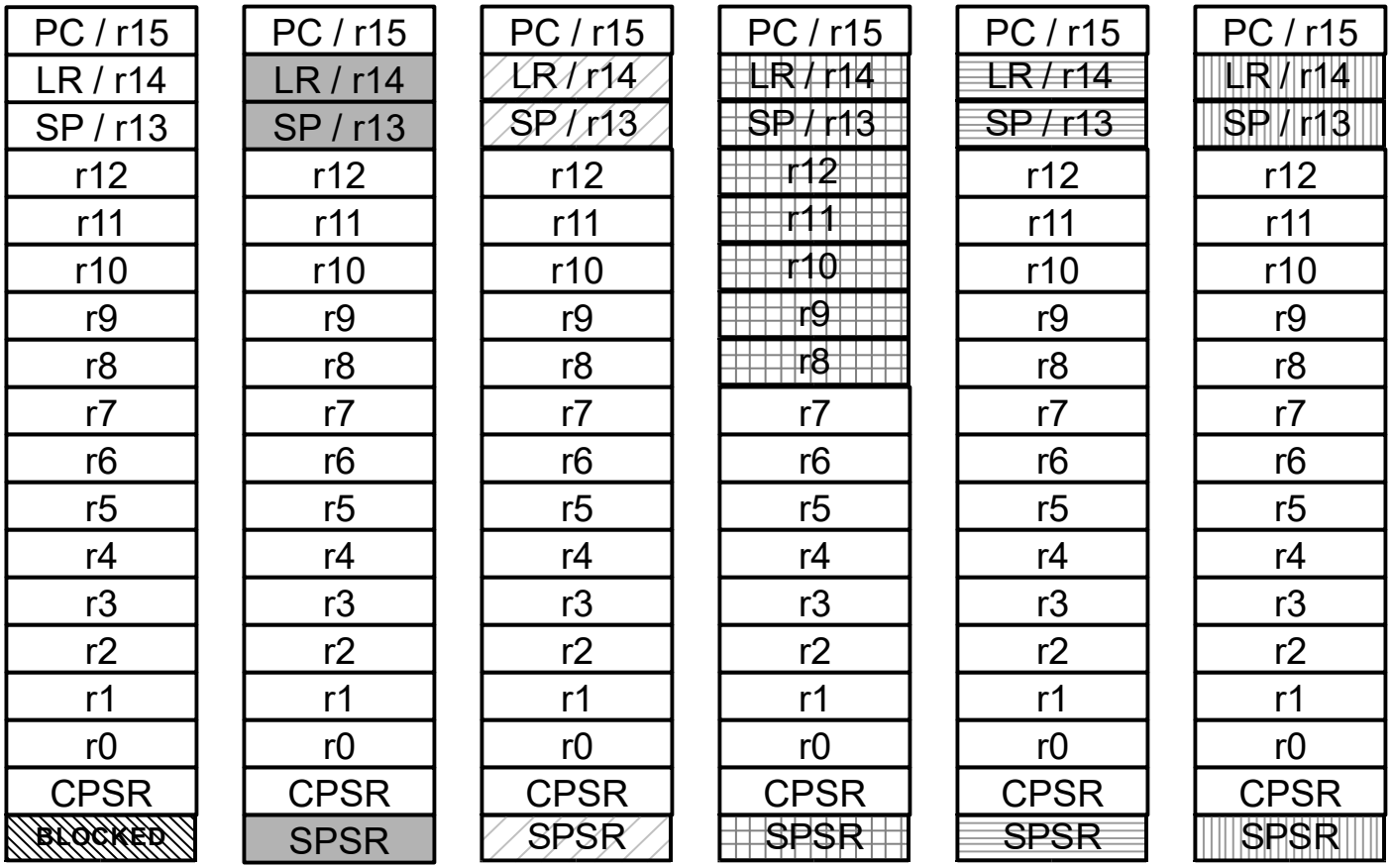
ARM Registers

- ARM has 37 x 32bit registers
- 1 PC (Program Counter)
- 1 CPSR (Current Program Status Register)
- 5 SPSR (Saved Program Status Register)
- 30 GPR (General Purpose Register) r0 to r14
- Only 15 GPRs are explicitly available to the programmer, the rest are used for swapping out data when changing modes (e.g., FIQ has its own registers, i.e. to save response time)

ARM Registers (cont.)

- Any mode can access:
  - A certain set of registers (r0-r12)
  - It's own stack pointer (r13 or sp)*
  - It's own link register (r14 or lr)*
  - The program counter (pc)
- All modes, except user and system has an SPSR (Saved Program Status Register)
- The sp and lr registers can be used as GPRs
- User and System mode share sp and lr

ARM Register Map

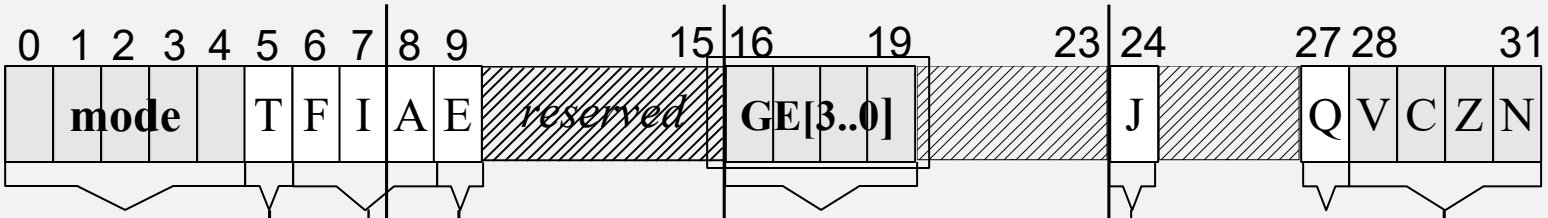


17 + 3 + 3 + 8 + 3 + 3 = 37 registers

The Program Status Registers (PSR)

- Only the CPSR is active
- SPSR holds value of CPSR immediately before the exception occurred

The Program Status Registers



Specifies processor mode

F=1 Disable FIQ
I=1 Disable IRQ
A=1 Imprecise aborts

N: Negative result from ALU
Z: Zero result from ALU
C: Carry result from ALU
V: oVerflow result from ALU

T=1 when processor in Thumb state

E=1 Big-endian
E=0 Little-endian

Q: Sticky overflow, used on newer cores

Used by Jazelle and other states to optimize access to bytes. Also used as extra flags.

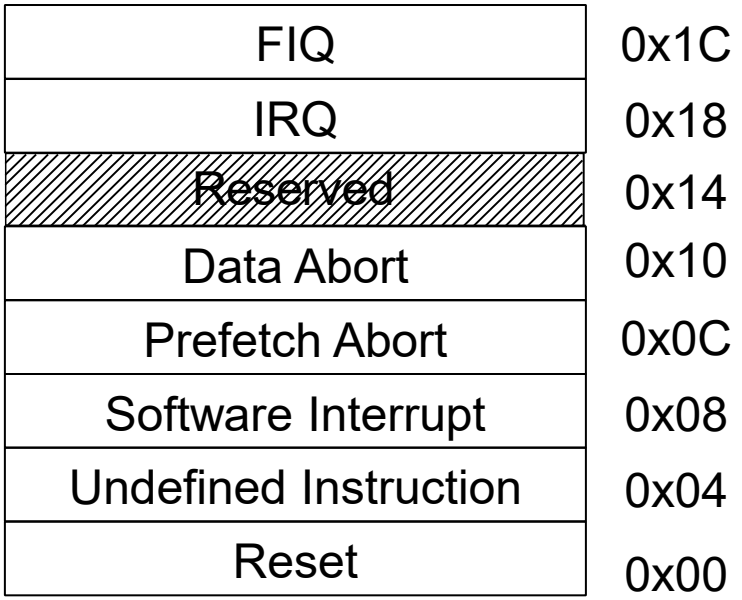
J=1 for Jazelle state

Program Counter Register

- Called r15 or pc in most assembler
- Use of pc depends on current state
- In ARM state:
  - PC values in bits 2..31 – bits 0 and 1 are ignored. Instructions are 32bit word aligned.
- In Thumb State:
  - PC values in bits 1..32 – bit 0 is ignored. Instructions are 16bits and half-word aligned.
- In Jazelle State:
  - Instructions are 8 bits, but processor reads 4 (32 bits) at a time.

ARM Exception Vector

- Just 32 bytes in size
- Starts at address 0x00000000 (can be set to 0xFFFF0000 for Windows CE)
- Each entry point to the vector contains an ARM 32bit instruction



ARM Exception Handling

- SPSR[mode] = CPSR (Save status register)
- Changes to relevant exception mode
- Changes to ARM state
- Disables Interrupts
- LR[mode] = PC (save return address)
- PC = Address of relevant vector entry
- Resume execution (of relevant vector entry)

Return from Exception

- Return from exception done by ISR:
- Change to ARM state
- CPSR = SPSR[mode]
- PC = LR[mode]
- Resume previous task

Endianness '*Switchianess*' of ARM

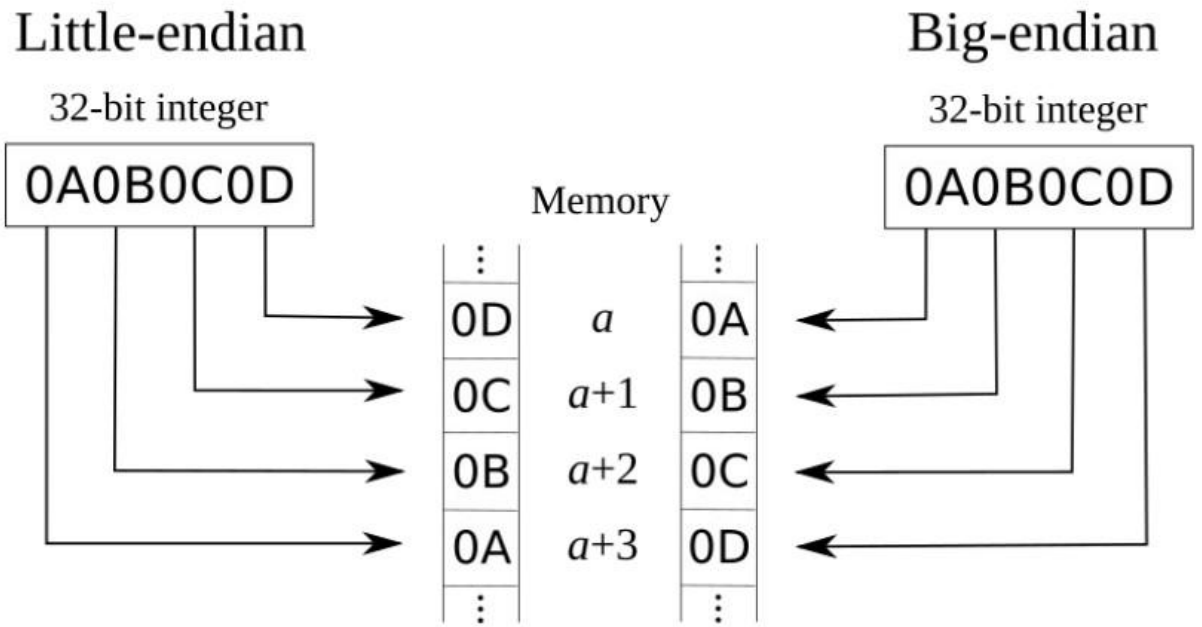
- Two ways to interpret contents of registers (and memory):
- Big Endian:
  - Most significant byte first (e.g. Motorola 68000, SPARC, et al.)
- Little Endian:
  - Least significant byte first (e.g. Intel x86 et al.)
- ARM supports **both** and can switch on the fly
- Can cause massive confusion…

  My advice: choose one and stick to it!! …

  (for the entire project and perhaps forever more?!)

Endianness (recap)

- Big Endian: Most significant byte first
- Little Endian: Least significant byte first
- ARM supports both and can switch



End of Lecture